Intermediate Submission 2 Group 58

Sofia Constantinou, Dimitar Zhekov, Nicole Almeida, Stefanos Kritikos, Alex Christou, Efe Koç
Eindhoven University of Technology, The Netherlands
2127326, 2132966, 2087480, 2153785, 2075407, 2098156, Group 58

1 INTERMEDIATE SUBMISSION 2

1.1 Activity Diagram

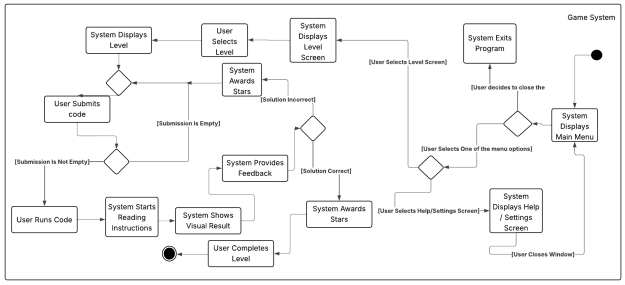

Figure 1: Activity Diagram

The activity diagram follows our main use-case description. The diagram starts by displaying the main menu. Based on the button pressed by the user, the system moves on accordingly. Regarding the main use-case, the system starts by displaying the level. When a submission is sent, the program checks whether the instruction log is empty or not. If an empty log is submitted, the user goes back and writes the instruction. If the log is not empty, the system moves on by running the instruction, visualizing the solution provided, and giving feedback. Stars are awarded if the solution is correct, and the level is completed. If the solution is wrong, the user has to re-write the instructions.

1.2 Testing Strategy

Table 1: Requirements Test Cases and Passing Criteria

Test Case	Passing Criteria	Linked Req
Implement unit tests to examine the movement of the character. Based on a simple generated 3x3 grid, tests will examine simple expected scenarios, like moving forward or validating the user's final position, but also examine unwanted behavior, such as the user trying to move through an obstacle or outside of the grid.	The unit tests will successfully if the system will behave in such scenarios as they are logically expected to. For example, in the tests where the character will try to go outside of the grid or through an obstacle, the system should not allow such movement.	1.4.1(18) The system must detect collisions with obstacles during executions, leading to the failure of a level.
Implement unit tests to examine level logic. The tests should be able to load the existing level progress of the user, check whether the completion process goes through accordingly, and that the progress is then stored in the progress file. Manual testing confirms level interactions respond within 3 seconds.	Unit tests should confirm level progress is restored correctly. Levels processed by the tests should be successfully stored in the properties file, and then any changes should be discarded after the test has passed. The next level of the one tested should be successfully unlocked.	1.4.2.(1) The system's settings and game progress must be saved and the modifications must be applied to the property file within 1 second of the user changing them.
Implement unit tests to verify the functionality of user code process management. The tests should verify that processes start successfully with valid class input, fail when class input is missing, and can be stopped correctly. The system must also verify that it accurately detects whether a process is alive.	The system should be able to start the user code process when a valid compiled class is provided. If the compiled class is missing, it should produce an appropriate error. The system must also be able to stop the process reliably and confirm whether the process is currently running or not.	1.4.1(11) The system must provide the user with the capabilities to run in-game Java commands that allow direct interaction with the sprite in a given level.
Implement unit tests to examine the functionality of the settings. The tests should verify the setting and retrieval of the avatar, as well as the volume levels, and ensure the persistence of the changes.	The selected settings should be persisted over different sessions, including the character, master volume, sfx volume, and music volume. Any changes should be reflected in the persistent file.	1.4.1(4) The system must provide the user with the ability to adjust the volume. 1.4.1(5) The system must provide the user with the ability to select their avatar from a predefined set.
Implement unit tests to ensure that commands provided by the user can be executed, paused, resumed and queue-cleared, without throwing any exceptions.	The system should be able to process a sequence of movement commands one step at a time. It should support pausing and resuming execution without throwing any errors. Commands must be cleared safely. The system should also maintain accurate pause state tracking.	1.4.1(15) The system must move the sprite step-by-step based on the instructions.